

Programmer en C++

1re STI2D · Cours · Variables, constantes, tests et boucles · Arduino ou ESP32

Un programme est la traduction d'un algorithme dans un langage. En C++, les idées de ce cours sont les mêmes sur un Arduino ou un ESP32 : seuls les numéros de broches et les tensions changent. Chaque instruction se termine par un point-virgule. Un bloc est entouré d'accolades { }. Le langage distingue les majuscules. Un commentaire commence par // et n'est pas exécuté.

1. ALGORITHME PUIS PROGRAMME

L'algorithme décrit la méthode, en français, sans détail de langage.

```
répéter sans fin
lire mesure
si mesure > seuil
    allumer
sinon
    éteindre
```

Le C++ est une écriture précise de ces étapes. L'ordinateur, puis la carte, exécutent exactement ce qui est écrit.

2. LES DEUX FONCTIONS DE BASE

```
void setup() {
    // une seule fois au démarrage
}
void loop() {
    // répété sans arrêt
}
```

void : la fonction ne renvoie pas de valeur.
setup prépare (sens des broches, liaison série). loop contient le travail permanent : lire, décider, agir.

3. VARIABLES

Une variable est un nom pour une case de la RAM. Elle a un type, un nom et une valeur. On peut changer cette valeur.

```
int n; // déclaration
n = 3; // affectation
int n = 3; // les deux ensemble
```

Un nom commence par une lettre.
Pas d'espace, pas d'accent. n et N sont deux variables.
Une variable déclarée dans un bloc { } n'existe que dans ce bloc.

LES TYPES UTILES

Type	Contenu	Exemple
int	entier	int n = 12;
float	décimal	float u = 1.5;
bool	vrai ou faux	bool ok = true;
char	un caractère	char c = 'A';
String	Chaine de caractères	String Mot="bonjour";

Le point décimal s'écrit avec un point: 3.3
Un texte affiché s'écrit entre guillemets: "bonjour".
Un caractère seul, entre apostrophes : 'A'.

4. CONSTANTES

const nomme une valeur que le programme ne doit plus modifier : un seuil, une broche, un maximum.

```
const int SEUIL = 25;
const int BROCHE_LED = 13;
const float U_MAX = 5.0;
```

On les écrit souvent en majuscules. Un seul endroit à modifier si le nombre change. Le compilateur refuse SEUIL = 30.

CONST OU #DEFINE

```
#define SEUIL 25
```

#define remplace le mot par 25 avant la compilation. Il n'y a ni type, ni point-virgule.
On préfère const : le type est visible, et une modification accidentelle est signalée.
Variable : en RAM, peut changer, perdue hors tension.
Constante : valeur fixe pour tout le programme.

5. AFFECTER, CALCULER, COMPARER

Le symbole = range une valeur : on calcule d'abord ce qui est à droite, puis on le range dans la variable de gauche. x = x + 1 augmente x de 1. Ce n'est pas une égalité mathématique.

Symbole	Rôle	Exemple
=	affecter	n = 3
==	égal à	n == 3
!=	différent de	n != 0
< > <= >=	comparer	n < SEUIL
&& !	et, ou, non	ok && !panne
+ - * /	calculer	s = a + b
++	ajouter 1	i++

DEUX PIÈGES

```
if (n = 3) // range 3 dans n
if (n == 3) // compare n à 3
```

Dans un test, il faut ==. Un seul = est une affectation.
Entre entiers, 5 / 2 vaut 2. Le reste est perdu. 5 / 2.0 vaut 2.5, car 2.0 est un décimal.
i++ est l'écriture courte de i = i + 1.

6. IF — CHOISIR

Le bloc est exécuté seulement si la condition est vraie. else traite tous les autres cas. else if ajoute un cas intermédiaire : seul le premier cas vrai est exécuté.

```
if (temperature > 25) {
    // chaud
} else if (temperature > 15) {
    // doux
} else {
    // frais
}
```

Pour deux cas seulement: if { } else { }. On met toujours les accolades, même pour une seule instruction.

7. FOR — UN NOMBRE DE TOURS CONNU

```
for (int i = 0; i < 4; i++) {
    // bloc répété
}
```

Zone	Quand	Ici
int i = 0	une fois, au début	i part de 0
i < 4	avant chaque tour	on continue ?
i++	après chaque tour	i augmente de 1

Tours: i vaut 0, 1, 2 puis 3. Quand i vaut 4, on sort. Quatre tours, pas cinq. i n'existe que dans la boucle.

8. WHILE — TANT QU'UNE CONDITION EST VRAIE

On l'utilise quand on ne connaît pas le nombre de tours à l'avance. La condition est testée avant le bloc.

```
int n = 1;
while (n < 100) {
    n = n * 2;
}
// ici n vaut 128
```

Si la condition est fausse dès le début, le bloc ne s'exécute jamais. Si rien ne la rend fausse, la boucle ne s'arrête pas.

LE MÊME COMPTAGE AVEC WHILE

```
int i = 0;
while (i < 4) {
    // bloc répété
    i++;
}
```

Même résultat que le for : i de 0 à 3. Pour compter, le for se lit mieux, car le compteur est dans l'en-tête.

for : je sais combien de fois.

while : je répète jusqu'à un changement. Sur une carte, il faut relire l'entrée dans la boucle, sinon la condition ne change pas.

9. ATTENDRE UNE MESURE

```
int mesure = analogRead(A0);
while (mesure < SEUIL) {
    mesure = analogRead(A0);    // sans cette ligne, mesure ne change pas
}
```

La boucle se termine quand la mesure atteint le seuil. A0 est la première entrée analogique d'un Arduino Uno ; sur un ESP32, on écrit un numéro de broche. delay fige tout le programme pendant l'attente.

10. Exemple

```
const int BROCHE_LED = 13;
const int SEUIL = 500;
void setup() {
    pinMode(BROCHE_LED, OUTPUT);
}
void loop() {
    int mesure = analogRead(A0);
    if (mesure > SEUIL) {
        for (int i = 0; i < 3; i++) {
            digitalWrite(BROCHE_LED, HIGH);
            delay(200);
            digitalWrite(BROCHE_LED, LOW);
            delay(200);
        }
    }
}
```

Au-dessus du seuil, la LED clignote trois fois. La broche 13 est celle de la LED intégrée sur la plupart des Arduino Uno ; sur un ESP32, on indique la broche câblée.

À RETENIR

1. const fige un nom. Une variable peut changer.
2. = affecte. == compare. La division d'entiers perd le reste.
3. if choisit. for compte. while attend.
4. i part de 0 et i < 4 : quatre tours, i de 0 à 3.

PIÈGES FRÉQUENTS

- Écrire = au lieu de == dans un if
- Oublier d'augmenter le compteur dans un while
- Oublier de relire une entrée dans le while
- Compter un tour de trop avec <=
- 1 / 2 qui vaut 0, au lieu de 1 / 2.0
- Vouloir modifier une const
- Oublier le point-virgule ou une accolade